ROAD SURFACE INTELLIGENCE

Road Damage Detection and Classification Using YOLO11n

Technical Project Report

Computer Vision • Deep Learning • Object Detection • Streamlit

Model: YOLO11n
Dataset: RDD2022 – India subset
Framework: PyTorch and Ultralytics
Repository: github.com/Parth7334/RoadSurfaceIntelligence

1. Abstract

Road Surface Intelligence is a computer vision project that detects and classifies visible road damage in images. It uses a YOLO11n object detection model trained on the India subset of the Road Damage Detection 2022 (RDD2022) dataset. The system supports longitudinal cracks, transverse cracks, alligator cracks and potholes. The workflow covers dataset auditing, annotation validation, YOLO-format conversion, exploratory data analysis, model experiments, evaluation and a Streamlit inference application.

The best-performing evaluated configuration achieved **30.3% mAP@50** and **12.9% mAP@50–95** on the validation dataset.

2. Problem Statement and Objectives

Manual road-surface inspection can be time-consuming and difficult to scale. This project explores deep-learning-based object detection to identify and localize common road damage categories in images.

Objectives

- Audit and validate image annotations.
- Convert selected damage categories into YOLO format.
- Train and evaluate YOLO11n.
- Compare dataset/training configurations.
- Build an interactive image-inference application.

3. Dataset and Preparation

The project uses the India subset of RDD2022. The dataset was audited and XML annotations were validated before conversion to YOLO format.

Audit item	Count
Training images with matching XML	7,706
Test images	1,959
XML annotations	7,706
Original object annotations	8,203
Missing image/XML pairs	0
Invalid bounding boxes	0
Image/XML dimension mismatches	0

Target categories: D00 – Longitudinal crack; D10 – Transverse crack; D20 – Alligator crack; D40 – Pothole. The dataset is not included in the repository and must be obtained separately under its applicable terms.

The baseline YOLO dataset contained 5,320 training images and 1,330 validation images. The validation split was retained across the reported experiments.

4. Methodology and Model Configuration

The pipeline consisted of dataset extraction, XML auditing, annotation validation, class-distribution and bounding-box analysis, target-class selection, YOLO-format conversion, model training, validation and Streamlit integration.

Parameter	Value
Architecture	YOLO11n
Task	Object detection

Parameter	Value
Input image size	512 × 512
Batch size	2
Training hardware	Apple M3 using MPS
Baseline / Experiment 1	40 epochs
Experiment 2	80 epochs

5. Training Experiments

Baseline: Original filtered training configuration, 5,320 images, 40 epochs.
Experiment 1: Retained all annotated training images and sampled 25% of empty training images; 2,959 training images, 40 epochs.
Experiment 2: Added 494 mixed target/non-target images to Experiment 1 while retaining the reduced empty-image subset; 3,453 training images, 80 epochs.

Configuration	Precision	Recall	mAP@50	mAP@50–95
Baseline	52.9%	24.9%	23.8%	9.94%
Experiment 1	42.7%	27.8%	25.4%	11.1%
Experiment 2	46.2%	30.1%	30.3%	12.9%

Experiment 2 produced the strongest validation results among the evaluated configurations. However, it changed both dataset composition and training duration, so the gain cannot be attributed independently to either change. In mixed-label images, non-target annotations were removed while visible non-target defects remained; these may be treated as background during training.

6. Class-wise Validation Performance

Damage category	Precision	Recall	mAP@50	mAP@50–95
Longitudinal crack	38.3%	19.7%	22.4%	9.23%
Transverse crack	43.7%	6.25%	7.16%	4.05%
Alligator crack	58.5%	60.5%	58.9%	26.5%
Pothole	44.3%	33.8%	32.8%	11.8%

The model performs best on alligator cracks. Transverse crack detection remains a major limitation because this category has very few training examples.

7. Confidence Threshold Evaluation

Confidence threshold	Precision	Recall
0.10	46.2%	30.1%
0.25	64.2%	28.0%
0.35	71.9%	20.4%
0.50	57.0%	9.85%

The application uses 0.25 as its default confidence threshold. This is a practical starting operating point, not a threshold proven optimal on an independent calibration dataset. The headline mAP values above are from the standard best-checkpoint validation results; confidence-filtered threshold tests are separate operating-point checks.

8. Application

A Streamlit application provides image upload, batch inference, original-versus-annotated image comparison, class labels, confidence scores, bounding-box coordinates, a combined detection table and CSV export. Separate tabs display class-wise validation performance and project information. The inference device is selected automatically from Apple MPS, CUDA or CPU, depending on availability.

9. Limitations

- Overall validation mAP is modest and does not establish production readiness.
- Transverse crack performance is weak due to class imbalance and limited examples.
- Unlabeled non-target defects in mixed-label images may introduce label noise.
- False positives and bounding-box localization need further assessment.
- Validation results do not establish performance on independent real-world deployment images.
- Experiment 2 changed multiple variables, limiting causal interpretation.

10. Future Improvements

Collect more transverse-crack examples; address class imbalance with controlled sampling and augmentation; preserve or explicitly model non-target annotations; run experiments that change one variable at a time; evaluate on an independent representative test set; and improve localization and false-positive control.

11. Conclusion

Road Surface Intelligence demonstrates an end-to-end computer vision workflow from dataset preparation and annotation validation through model experimentation, evaluation and application development. Experiment 2 was the strongest of the tested configurations, achieving 30.3% mAP@50 and 12.9% mAP@50–95 on the validation set. The project also documents important challenges in class imbalance, annotation quality and generalization.

12. Repository

GitHub: https://github.com/Parth7334/RoadSurfaceIntelligence
The repository includes the Streamlit application, prediction script, trained model, dataset preparation and experiment scripts, sample images, requirements and README. The full RDD2022 dataset is not included.

Appendix A. Application Screenshots

Dashboard overview

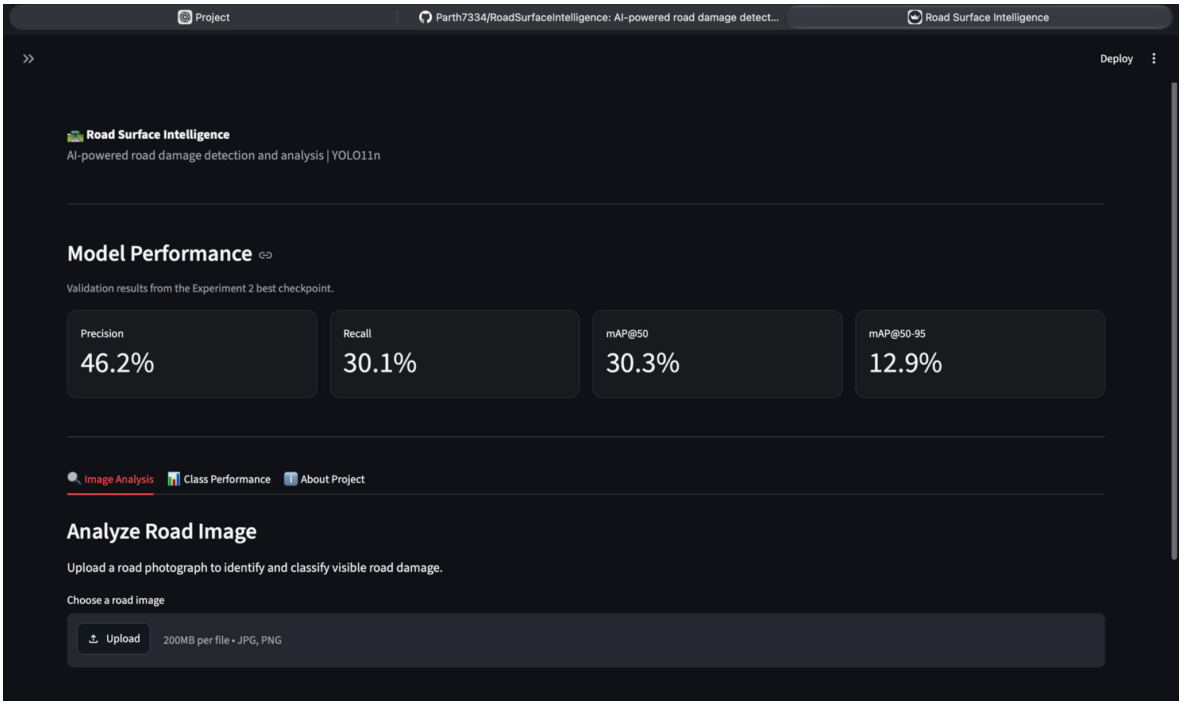


Figure A1. Streamlit dashboard and model performance summary.

Image prediction and detection report

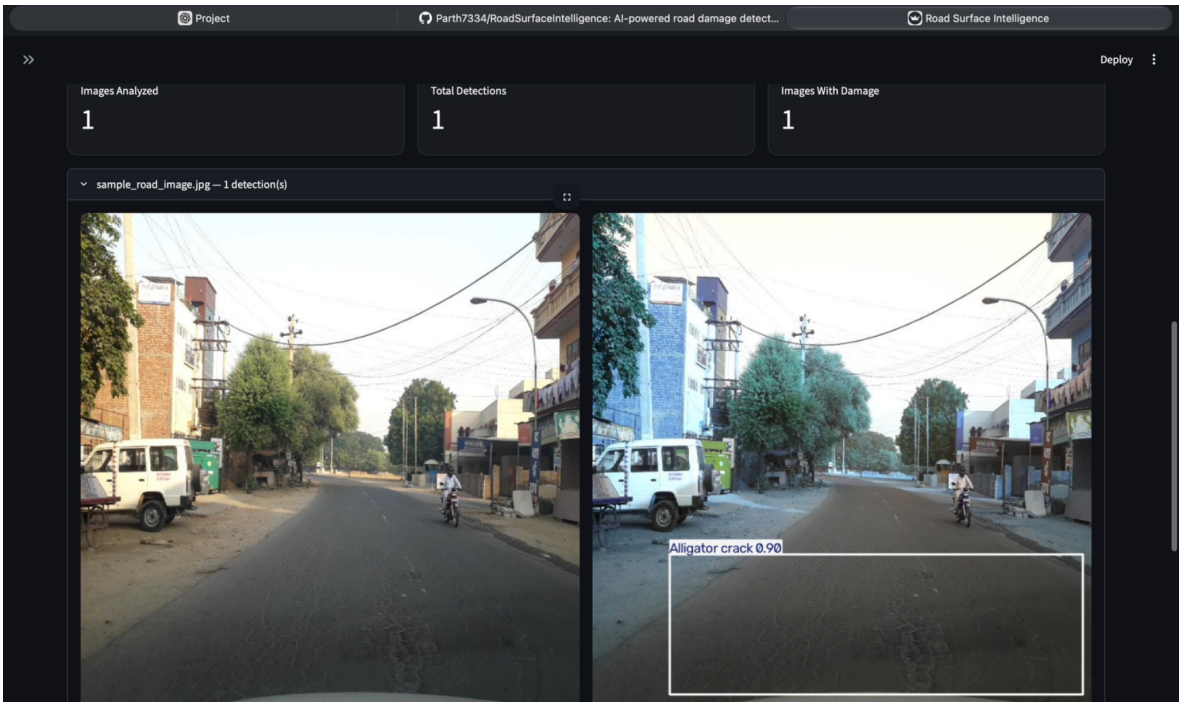


Figure A2. Original and annotated road image with an alligator-crack detection.

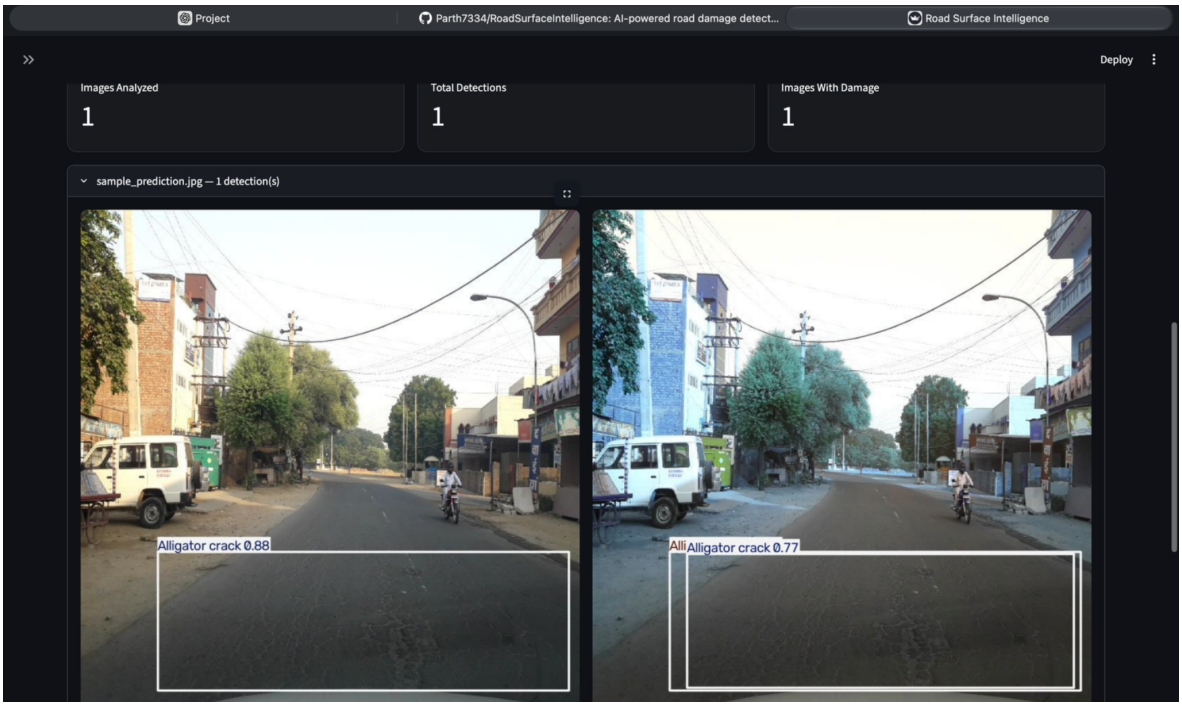


Figure A3. Second sample inference result. The uploaded image is already annotated; this is a demonstration, not an independent evaluation.

Class performance and project information

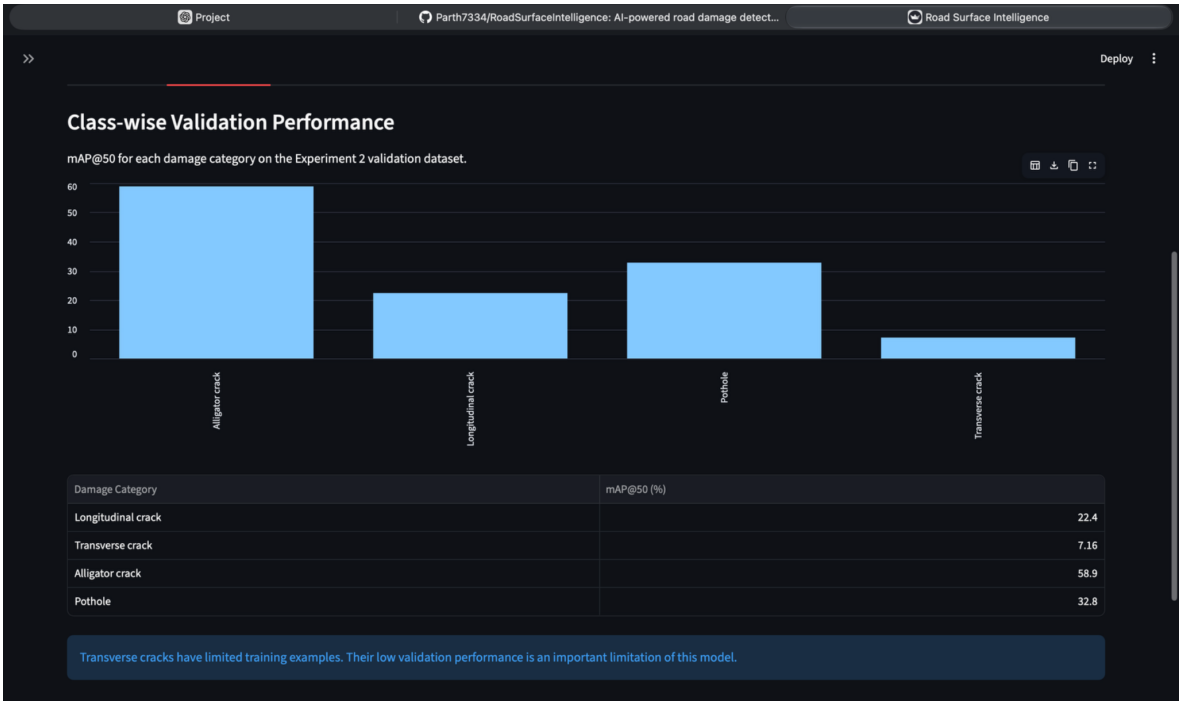


Figure A4. Class-wise validation mAP@50 chart and metrics.

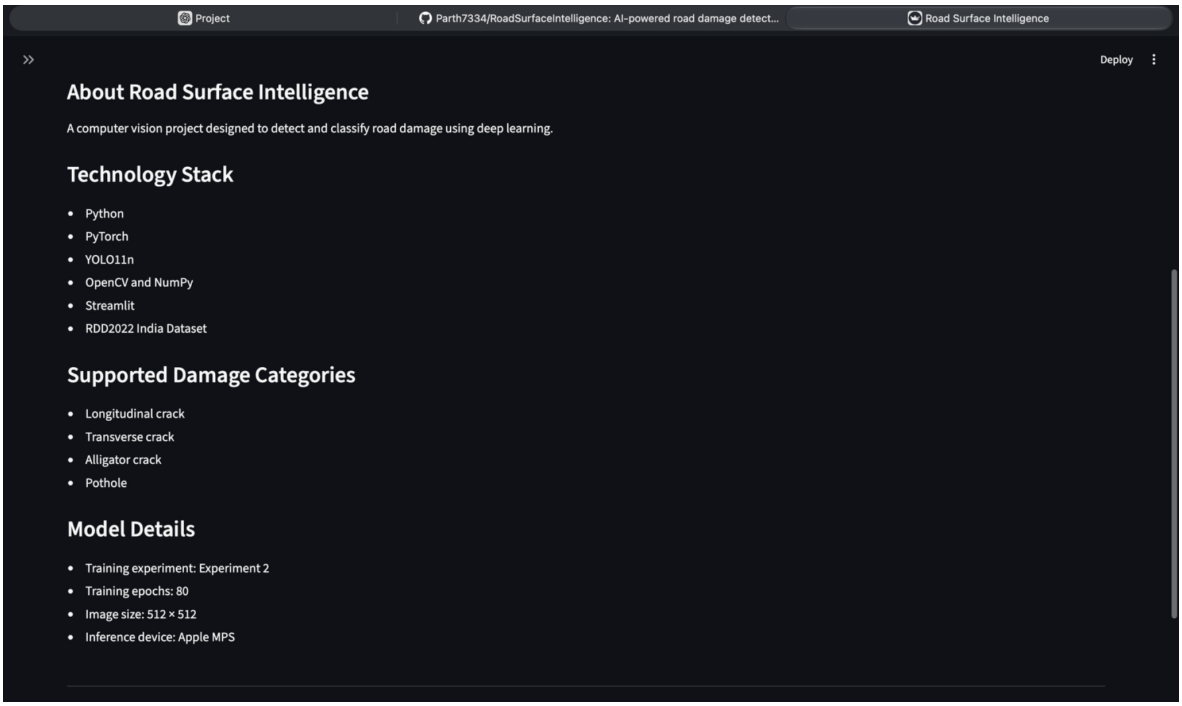


Figure A5. About section with technology stack, supported categories and model details.